Aadhaar App Intent Integration OPENID4VP Specification v1

Version	Version Date	Created by	Reviewed by	Approved by	Status
v1	17-April	Shaikh Roshan	Abee Narayan		Testing

Aadhaar App Intent Integration OPENID4VP Specification v1 1

1. Introduction 3

2. Before You Start 3

2.1 Cryptographic Setup 3

2.2 OVSE Registration 4

2.3 Backend Requirements 4

3. Steps for Presentation Request Flow 5

3.1 Create Authorization Request 5

3.1.1 JWT Header 5

3.1.1 Request Parameters 5

3.1.2 Response Structure 6

3.1.3 Field Specification 6

3.2.1 JWT Header 6

3.2.2 JWT Payload Structure 7

3.2.3 Payload Field Specification 7

3.2.4 Presentation Definitions 8

3.3 Choose Invocation Method 13

3.3.1 App to App 13

3.3.2 Web to App 13

3.3.3 Cross Device QR 13
3.4 Receive SD-JWT 14
3.5 Verify Presentation 16
3.5.1 Signature Verification 16
3.5.2 Presentation Validation 16
3.5 Send Acknowledgement 16
3.6 Error Codes and Failure Scenarios 17
3.8 Security Requirements 18
3.9 Final Integration Checklist 18

1. Introduction

This document provides a complete end-to-end guide for integrating with Aadhaar App using OpenID4VP (OpenID for Verifiable Presentations).

The integration enables Verifiers to request and receive Verifiable Credential presentations from Aadhaar App using standardized OpenID flows.

The integration supports three invocation mechanisms:

1. App to App flow
2. Web to App flow
3. Cross Device QR flow

The request object (JWT) structure remains common across all flows. Only the invocation method differs.

2. Before You Start

Before beginning integration, ensure the following prerequisites are completed.

2.1 Cryptographic Setup

You must generate an RSA key pair.

1. Private key → used to sign OpenID request object
2. Public certificate → embedded in x5c header

Requirements:

1. Algorithm: RSA
2. Signing: RS256
3. Private key must remain in backend
4. Private key must never be embedded in frontend or mobile apps

2.2 OVSE Registration

To obtain Verifier Client ID, you must provide:

1. Public certificate (x509)
2. Domain name
3. Callback / redirect URI
4. Entity display name

UIDAI will register your verifier and issue client identifier.

Integration cannot proceed without registration.

2.3 Backend Requirements

Your backend must support:

1) JWT creation and signing (RS256)
2) x5c certificate embedding
3) HTTPS endpoints
4) Request state management (state, nonce)
5) VP token parsing and verification
6) Base64URL encoding

2.4 JWT Signature Validation (Recommended Pre-check)

Before sharing the public key and proceeding with integration, OVSEs are strongly recommended to validate the JWT signature format and structure.

Before integration:

1. Generate sample request JWT
2. Validate using public certificate
3. Ensure:
   a) RS256 signature valid

b) x5c chain correctly embedded
c) Claims match specification

(This is for development validation only and should not be used in production flows.)

3. Steps for Presentation Request Flow

3.1 Create Authorization Request

Every integration begins by creating an authorization request.

Endpoint:
POST `/samvaad/openid/create-request`

3.1.1 JWT Header

```
{
  "alg": "RS256",
  "typ": "JWT"
}
```

Algorithm must be RS256.

3.1.1 Request Parameters

Field	Type	Allowed Values	Mandatory	What You Must Do
acr_values	String	urn:uidai:acr:face, fingerprint, iris, pin, mfa	No	Default to face if not provided

3.1.2 Response Structure

```
{
  "request_id": "string",
  "request_uri": "string",
  "qr_code_url": "string",
  "acr_values": "string",
  "expires_at": "timestamp"
}
```

3.1.3 Field Specification

Field	Type	Format	Mandatory	What You Must Do
request_id	String	Alphanumeric	Yes	Store for correlation
request_uri	URL	HTTPS	Yes	Fetch request
qr_code_url	String	URL encoded	Yes	Openid, client, request uri
acr_values	String	URN	Yes	Must match requested auth type
expires_at	Timestamp	ISO UTC	Yes	Enforce expiry (5 min)

3.2 Get Request Object (JWT)

Endpoint:

```
GET /samvaad/openid/request/{id}
```

3.2.1 JWT Header

```
{
  "alg": "RS256",
  "typ": "oauth-authz-req+jwt",
  "kid": "key-id",
```

```
  "x5c": ["cert", "ca-cert"]
}
```

3.2.2 JWT Payload Structure

```
{
  "iss": "x509_san_dns:domain",
  "aud": "https://self-issued.me/v2",
  "client_id": "x509_san_dns:domain",
  "iat": 1700000000,
  "exp": 1700000300,
  "nbf": 1700000000,
  "state": "string",
  "nonce": "string",
  "acr_values": "urn:uidai:acr:face",
  "presentation_definition": {}
}
```

3.2.3 Payload Field Specification

Field	Type	Format	Mandatory	What You Must Do
iss	String	x509_san_dns	Yes	Must match certificate
aud	String	URL	Yes	Must be self-issued.me
client_id	String	x509_san_dns	Yes	Same as issuer
iat	Integer	Unix time	Yes	Current timestamp
exp	Integer	Unix time	Yes	Short expiry (5 min)
nbf	Integer	Unix time	Yes	Set equal to iat
state	String	Random	Yes	Maintain session mapping
nonce	String	Random	Yes	Prevent replay

acr_values	String	URN	Yes	Authentication method
presentation_definition	Object	JSON	Yes	Define requested claims

3.2.4 Presentation Definitions

Presentation Definition Structure (Reference)

```
{
 "id": "aadhaar_vp_request",
 "input_descriptors": [
  {
   "id": "aadhaar_credential",
   "format": {
    "jwt_vc": {},
    "sd_jwt": {},
    "mso_mdoc": {}
   },
   "constraints": {
    "fields": [
     {
      "path": ["$.credentialSubject.ResidentName"],
      "purpose": "User identity verification"
     }
    ]
```

```
    }
   }
  ]
}
```

Bitmap to Presentation Mapping Table

Claim Name	JSON Path	Presentation Definition Mapping	Mandatory	What You Must Do
CredentialIssuingDate	$.credentialSubject.CredentialIssuingDate	constraints.fields.path	No	Include if issuance validation required
EnrolmentDate	$.credentialSubject.EnrolmentDate	constraints.fields.path	No	Include if enrolment validation needed
EnrolmentNumber	$.credentialSubject.EnrolmentNumber	constraints.fields.path	No	Use only if explicitly required
ResidentImage	$.credentialSubject.ResidentImage	constraints.fields.path	No	Avoid unless

				KYC required
ResidentName	$.credentialSubject.ResidentName	constraints.fields.path	Yes	Must be requested for identity
LocalResidentName	$.credentialSubject.LocalResidentName	constraints.fields.path	No	Optional for regional display
AgeAbove18	$.credentialSubject.AgeAbove18	constraints.fields.filter.const	No	Prefer over DOB for privacy
Dob	$.credentialSubject.Dob	constraints.fields.path	No	Use only if exact DOB needed
Gender	$.credentialSubject.Gender	constraints.fields.path	No	Optional
CareOf	$.credentialSubject.CareOf	constraints.fields.path	No	Optional
Building	$.credentialSubject.Building	constraints.fields.path	No	Address validation

Locality	$.credentialSubject.Locality	constraints.fields.path	No	Address validation
Street	$.credentialSubject.Street	constraints.fields.path	No	Address validation
Vtc	$.credentialSubject.Vtc	constraints.fields.path	No	City/Town
District	$.credentialSubject.District	constraints.fields.path	No	Regional validation
State	$.credentialSubject.State	constraints.fields.path	No	Regional validation
Pincode	$.credentialSubject.Pincode	constraints.fields.pattern	No	Must match 6-digit format
Address	$.credentialSubject.Address	constraints.fields.path	No	Full address
Mobile	$.credentialSubject.Mobile	constraints.fields.pattern	No	Must match +91 format
MaskedMobile	$.credentialSubject.MaskedMobile	constraints.fields.path	No	Preferred over full mobile

Email	$.credentialSubject.Email	constraints.fields.format	No	Optional
MaskedEmail	$.credentialSubject.MaskedEmail	constraints.fields.path	No	Preferred

Mapping Rules

- Each bitmap bit set to **1** must correspond to a `constraints.fields` entry
- Each field must include:
  - `path`
  - Optional `purpose`
  - Optional `filter` (for validation)
- Example with multiple claims:

```
{
 "constraints": {
  "fields": [
   {
    "path": ["$.credentialSubject.ResidentName"],
    "purpose": "Identity verification"
   },
   {
    "path": ["$.credentialSubject.AgeAbove18"],
    "filter": {
     "type": "string",
     "const": "Yes"
    }
   }
  ]
 }
}
```

3.3 Choose Invocation Method

3.3.1 App to App

Launch using OpenID4VP deeplink:

Android:

```
val intent =
Intent("openid4vp://?client_id=<client_id>&request_uri=<request_uri>")
.apply {
    putExtra("request", jwt)
}
startActivity(intent)
```

iOS:

```
UIApplication.shared.open(URL(string:”openid4vp://?client_id=<client_id>&request_uri=<request_uri>")!)
```

3.3.2 Web to App

```
window.location.replace("openid4vp://?client_id=<client_id>&request_uri=<request_uri>";end");
```

3.3.3 Cross Device QR

Steps:

1. Decode `qr_code_url`
2. Generate QR
3. User scans QR using Aadhaar App

4. Aadhaar App resolves request_uri

3.4 Receive SD-JWT

The wallet sends response to verifier backend (call back url).

Expected response:

```
{
  "vp_token": "JWT/SD-JWT/mDoc",
  "presentation_submission": {},
  "state": "string"
}
```

Format:

SD_JWT

eyJhbGciOiJSUzI1NiIsImtpZCI6InVpZGFpLTM3YThlNzQyLWMzNzUtNDQzYy04Y2U1LThiN2
EwODIyY2I2YiIsInR5cCI6InZjK3NkLWp3dCJ9.eyJpc3MiOiJodHRwczovL3BlaGNoYWFuLnVp
ZGFpLmdvdi5pbiIsImV4cCI6MTgwNTQzNjE2MSwiaWF0IjoxNzczOTAwMTYxLCJpZCI6IjAzM
jRmNTlkLWUxNjItNDQzMy1iYjg5LWYwNjY0NDljMGM5ZCIsImNuZiI6eyJqd2siOnsiY3J2IjoiU
C0yNTYiLCJrdHkiOiJFQyIsIngiOiI5dS1SNkZYRS13eURRTG5xWHBrQXoxOXVzUE1YM3VUR
FZQeGVJUnhIQU5VIiwieSI6ImFSMTdkcTRfN2hFdEtaa3I3c1p4ZExBN1BKWllwbFdieWE1OX
ktSkVfTnMifX0sIl9zZF9hbGciOiJzaGEtMjU2IiwiX3NkIjpbImQtMVpBcTdUUVVPMk5TM1M4a
GxJaVk1RVFmbFcyMHlNUERVRWxmcDdWQzAiLCJvQWVOS19Xanp2OFU3MjA4cjJ5U2pa
WVgwR2tjblVCOGc5ZVdrTWFXR3RzIiwiY01YQUJKZ1E0UGZVeVZ5c0JLallrQmpraUhqWEV
SeE1qemc4SlBMMV92VSIsImxVNG5hLVNCNm1vY1BYOGdpUUs1YW1aQUs2aFNvZzRFS3
g0Z1I1U0YwWEEiLCJWd2Z1TnFlUXRMaW5NUHRCYjg4NExhZ1lycFFhZGNSSHFmM2tjaXlt
U2p3IiwiSUdNSUdXLWRHd2JMRzB0LVo5Z0VPUXZDZ3VqTHIwT0txUXQydU1rX0xUYyIsImt
TczNyOXJvb2E2NFZyNy1DRXpvUk8xcExXVkZCR3JLNy11bXZMZTQyOTAiLCJkU1ZoMl9RNm
N5TWRsV2p2enpBVUNveDAxdlI2bGI0RU9MdFQwQVNxajhJIiwiSXJJNWxYMk5yNXgtbV9DV
U4wbXZSTVlXSi1QdTVQWHN4NWtwVjVoZFRFZyIsInQ0dENocGFIdko0V0d3M2JiOEs0T3Za
aG1GQXE2cWJYVDQtZVdCa0JmSW8iLCJxbnlqUFN4Y3NDWVR6REQtbndzT1R0eHpsb0d1
N3l1aDV2ZnJmMkhteUtRIiwiekVOamJyWGRvVlBpWmdhWkZ2N2tydm1GRlZyVFpyV0Q0Yk

JZajVIRzdsNCIsImxHbGpEVlE2V0E5Z2paV1FPTEVRQVBtNkVUT3U5dEJLakVQRG9DNGtGc
W8iLCJqR1o3ZGNDU3lzV1BYTXA0eW80ZnUybVg0UTd5a1R1TXJxWnhRTzhXdHdRIiwiUGw
xZ0tybl8zOVJBZ3J2WXAxeGRJZjZRTTdUMm1SOUwzNUZDZ0cwUlBKayIsIk9LTGZYSW9SajF
uSG1GZWVxNks3TmdTenl5blh2dHJIMk9rWTB1eGxnTzAiLCJVeUpMb0d6aDRhQUdCV0gx
NExQMjhGLU5zajVHYl80cW5BWExkbG9paGp3IiwiMHpXNDhXNC1tXzhZOVJNVTNtUUxfR
3IwdVpQV2VKMUZ3N080V19nZmxyTSIsIlVia0o3ZG1peDdhc1EzcVdRb1hDRVhkcmo1bFZl
MlhjOXhqMnQ1ckRyWW8iLCJqNllzaHBnVFRldjJfeGNyQ1dwVUR0dzNURHpUNG94YWhp
WXF4NWNOSW1ZIiwiNm03UlRpM0dIczhJVXhVNFlVajJUMy1EU1l0XzNyc1pkZFIzVDJmQX
A2ayIsIktYWUVUWWMtNEdqOTl3T0loTmpVT0MxSVEteFNHeGg2S2hMaHpGS3kwNE0iLCJ
4THFSX1IwYVk4azQ2QjdIZ1U2a3pZemZVZEZRZFlmeWRuTDZsRzRhc2JVIiwieENZSEZ4bXB
KZ0JrV0NHLXgzU294QXRuQS1SallsN01pOXY4Q1ZfeVI2WSIsIlg0RkFuaGNRek5ady13aUR
yR2lrVk9kVHhicVJtUXhZaGVPSW1WNjNTTzQiLCI1ZFE0d051MnFYQ2NkUFBLRl9yQ0kzSG
Y1dHpDbEgwQ2lVT1JUUWhYUlFZIiwieVpJUzN2VkJibXVWNzlZQlFXWkFRZWNuVmlDMm5
1ekJJZE1JeUFXM2FTMCIsImlXcUlSNVE0Wmh0UE5pdWhhdVlkVVZkSkYxOUlXVlpUUlJGTEx
GVmY2WnciLCJubE1FV01QZ1pRZDRhdjdpYWhidzhzRHB1NnZ2TE5BdE5ZZ3V5ejBZblVVIi
wiX2lybF9NT2tBOWFpOE1uU3dZMUZ2cFN3QmdETXZYajNrTFZ0NklabndvcyIsIm45R2Zhe
UNjUWJRcUxRQjRoZjV0ZWMxZjFpTWZ0TlBFbTRVN2MxQkhaQmMiLCJLTHp5ZFRMTnQ4VT
Rqb0VLRjhwVmhWU21fVUY5dS1EamdpV2RvQzBpNEhBliwiZTc4b25uQmhaUk9LenItWlFZ
LTh4RjNObjdoc19Ta182Z3dJZE5hNDEwbyIsIm5tdGg5c1B4SEN2Yk9ENkRvUjFsalNydWtO
c0o4MS1Mb1Q3Q09XQ3FHbDQiLCJxYTQ5czVUdmg2VllRLWZ5Q2VBR3RIM1RQOWN0YTh
0YlkxNFZqaEllYUZIiwiZTcxVkhtQVVHSE1JWUtwVmNaRVFmVzBfVFRlcXk2OTF5M0R0RjNW
RUREZyIsIlBhTTJzU0NuSkdnVVhuME1wYmhSV25FcEZaMGJMTzdVbnVFU3JSazhXVkUiLCJ
0WmZrT0Z1OHZoR2tlczFGd0pKblFlVjA4TWFaVjd3MHhTdHYteDdSMERFIiwiNFpUSTQxS0
E1cF9NNTVIbWRlRzZSd2lfNXVNcUZCRTk1dEpBcExWLU5sOCIsIjNFWUxxTXdacmo2MnN
STXRVaktzQkdNVWZ5Yk5jWUN2QjZxQUVYM0FoUnciLCI5dm9uYmRJT21MSnhqZUlWMHJ
wek5zQU1iTlpwYmNNWkI2MWRjWG93eGZjIiwiSkxtSW9fdzhrc00xdllqSlVjVEVMbGFPOTZ
obzQwdHpWODYxNnVUVWNwRSJdfQ.ODWh9TJ8cvXTMu2GD7B1v7EYLiBUOhaLkAxLsUY
viVZbk39sdsWbi73qrwQCJNNAWh90Dc5yD849zpWxBv2BlAlYC_fCuJGz34QKUsoecgAjBo
GekJ41lUEVoVpEYnnuug-
oB0yHfFpl8xRY0IqvlrkrozX3A9hMliZ1W61C3Wi16qKZYO150qJQcy9XPfGSqBSvV6JN_LWZE
rKunzmhYsML17LHkeN3AFpmESS-
AM5QgNxy784uzcvHTv77fO4z6mGrywG_lS9Bdw19mIP-ndynqg_htQfyO-
QpteXCzMLGmIfy2lp39l-s-eLiq70lfCzEeJmQF1yEbZ8a6u6ltLnktodiGYy0BI69HsYkY-
qv88iBi8axM9bQhG3NUQ1PbqLtv92tsE3GjlyUcmd9BQWkDUEqqLNhdGXZex_bwoClYDjc
2XjCpqjMCuYjRMLOYv9ozXB9zhRzVSkrOMLrstYVQV_vQsavQOLQ37N5XuTAyJBm6cBgE7y
71M1h00O4eyUPhDP0aXBkV_1Syls3eoe4cvKg2Z4GvUiNSWf7qHU_c5XzagtXTEKTtxzsUarZ

GmsP-
MkSiqq2l0JmviZaA97NLwtFNS9E4Phl9UyCg9ZWNo3NRGiXMO6pXZlZWeTMX6e0j3tE7AO4
9OrdULDwl_tawFoEWZ2-
2qHOwfTgiHo~WyJjNjUzMmYzMi0yZTViLTQxYjMtYWM5MS01NjYzYmZhYTQ1YmQiLCJSZX
NpZGVudE5hbWUiLCJBYmVlIG5hcmF5YW4iXQ

3.5 Verify Presentation

3.5.1 Signature Verification

1. Extract vp_token
2. Identify credential format
3. Retrieve issuer public key
4. Verify signature
5. Reject if invalid

3.5.2 Presentation Validation

For each disclosure:

1. Validate nonce matches request
2. Validate state matches session
3. Validate presentation_definition compliance
4. Validate credential expiry
5. Validate revocation status (if applicable)

All validations are mandatory.

3.5 Send Acknowledgement

```
{
  "request_id": "string",
```

```
  "response_code": 200,
  "response_msg": "Success"
}
```


For failure scenarios:


```
{
 "request_id": "string",
 "response_code": "<ERROR_CODE>",
 "response_msg": "<ERROR_MESSAGE>"
}
```


3.6 Error Codes and Failure Scenarios

The following error codes standardize failure handling across Aadhaar App authentication flows. OVSE systems must handle these appropriately and provide meaningful user messaging.

Name	Case	Error Code	Error Message
User Reject	User explicitly taps Deny / Reject	301	User rejected the authentication request
Face Liveness Fail	Face liveness detection fails repeatedly, exceeding retry limit	302	Face liveness verification failed after 3 attempts
User Abort	User exits flow before completion (back press, app switch, close)	303	Authentication process aborted by user
Face Mismatch	Face captured successfully but does not match Aadhaar records	304	Face authentication failed due to mismatch with registered data
Biometric Lock	Biometric lock enabled in UIDAI system	305	Biometric authentication is locked for this user

Commented [AN1]: or Auth FailACR authentication failed302Authentication failed

FaceRD Server Down	FaceRD service unavailable	306	FaceRD service is currently unavailable. Please try again later
HTTP Error	Network/API failure (timeout, 5xx, invalid response)	307	Failed to complete authentication due to network or server error
Callback Retry	Auth success but callback delivery failed	308	Authentication completed but callback delivery failed. Retrying
Invalid VP	Signature/format invalid	309	Invalid presentation token
Expired Request	Request expired	310	Request expired
Replay Attack	Nonce/state mismatch	311	Invalid or replayed response
Server Error	Backend failure	312	Server error

Commented [AN2]: To be Implemented

3.8 Security Requirements

1. Private key must remain backend
2. All endpoints must use HTTPS
3. exp must be short-lived
4. Validate state and nonce
5. Reject replayed responses
6. Validate certificate chain (x5c)
7. Do not log credentials

3.9 Final Integration Checklist

1. RSA key pair generated
2. Certificate configured

3. Authorization request working
4. JWT request object valid
5. QR / deeplink working
6. VP token verified
7. State and nonce validated
8. Error handling implemented